

Rapport de tests du backend

Vérification non-destructive de l'API REST `SmartFridge` — port 3001
Document de validation fonctionnelle (runtime)
Résultats réels obtenus lors de l'exécution des tests — aucun résultat inventé

1. Présentation du projet ROCT

SmartFridge (ROCT) est une application de gestion de réfrigérateur : l'utilisateur enregistre ses produits, surveille leurs dates d'expiration, reçoit des notifications automatiques, trace ses consommations et visualise ses pertes alimentaires. Le backend est une API REST Node.js / Express connectée à une base MongoDB (locale sur `localhost:27017/SmartFridge`) par l'intermédiaire de Mongoose.

2. Objectif des tests

- Vérifier que l'API REST existante fonctionne correctement, sans modifier aucun fichier du projet.
- Valider les parcours d'authentification, la gestion des produits, des catégories, des notifications et les statuts d'expiration.
- Vérifier les réponses HTTP, les erreurs attendues, l'authentification (JWT), la validation (Zod) et l'isolation entre utilisateurs.
- Confirmer que les statuts `valide`, `bientot_expire` et `expire` sont correctement calculés.
- Ne pas altérer les données existantes (base non supprimée, aucune `dropDatabase()`, uniquement des données temporaires clairement identifiées puis nettoyées).

3. Environnement / technologies testées

Élément	Valeur constatée
Backend	Node.js v24.10.0, Express 5, Mongoose 9
Endpoint	<code>http://localhost:3001</code> — réponse racine « SmartFridge Backend is running! »
Base de données	MongoDB locale <code>mongodb://localhost:27017/SmartFridge</code>
Authentification	JWT (jsonwebtoken), hachage des mots de passe (bcryptjs), Bearer token
Validation	Zod (<code>registerSchema</code> , <code>loginSchema</code> , <code>productSchema</code>)
Méthode de test	Appels HTTP directs (<code>Invoke-WebRequest</code>), script temporaire hors du projet ; aucun outil ajouté au projet
Statut du serveur	Fonctionnel — répond sur le port 3001

4. Tests réalisés par domaine

4.1 Authentification

- ✓ Register — création de compte (201, notification de bienvenue créée)
- ✓ Register (email en double) — 400
- ✓ Login — 200 avec JWT, identité correcte
- ✓ Profile — 200, identité et email corrects
- ✓ Update profile — nom (200) ; email sans mot de passe (400) ; email + mauvais mot de passe (401) ; email + bon mot de passe (200)
- ✓ Logout — 200
- ✓ Mauvais credentials — mauvais mot de passe (401) ; email inconnu (401)
- ✓ Requête sans token — 401 ; token invalide — 401

4.2 Produits

- ✓ Get products — 200, tableau
- ✓ Create product — 201 (3 produits de statuts différents)
- ✓ Erreur de validation — 400 (Zod)
- ✓ Get product by id — 200 ; id inconnu — 404
- ✓ Update product — 200, quantité mise à jour
- ✓ Consume product — 200, quantité décrétementée + montant consommé correct ; quantité 0 (400) ; quantité supérieure au stock (400)
- ✓ Delete product — 200, puis 404 à la relecture

4.3 Catégories

- ✓ Get categories — accès authentifié requis (401 sans token), 200 une fois authentifié
- ✓ Create / Update / Delete category — refus pour un utilisateur non-admin (403) : restriction admin opérationnelle
- Note constatée : les catégories sont un référentiel global, non scoped par utilisateur.

4.4 Notifications

- ✓ Get notifications — accès authentifié requis (401 sans token) ; 200 avec la notification de bienvenue
- ✓ Filtre par date — 200
- ✓ Marquer comme lue — 200, `isRead` passe à true

4.5 Statuts des produits

- ✓ `valide` — produit dont l'expiration dépasse le seuil d'alerte
- ✓ `bientot_expire` — produit dans la fenêtre d'alerte (`expirationAlertDays`)
- ✓ `expire` — produit passé

4.6 Validation et sécurité

- ✓ Validation Zod active sur `/api/auth/register`, `/api/auth/login` et `/api/products` (400 sur payload invalide)
- ✓ Mots de passe hachés (bcrypt), jamais renvoyés par l'API
- ✓ Middleware `auth` (401) et `admin` (403) vérifiés
- ✓ Accès admin refusé pour un utilisateur standard — `/api/admin/statistics` (403), `/api/admin/users` (403)

4.7 Tests du code-barres / Open Food Facts

- ✓ GET `/api/products/barcode/3017620422003` — 200 avec données produit (nom, etc.) via le service Open Food Facts

4.8 Isolation des utilisateurs

- ✓ Le second utilisateur (B) ne voit pas les notifications du premier (A)
- ✓ B ne peut pas marquer lue une notification de A — 404

- ✓ B ne peut pas lire (404) ni consommer (404) un produit de A

5. Résultats globaux

46	46	0	0
TESTS / CHECKS	RÉUSSIS	ÉCHECS	SKIPPED

Indicateur	Valeur
Total	46
Passed	46
Failed	0
Skipped	0
Backend status	WORKING

6. Points particuliers / limites

- Aucune suite de tests automatisés n'existe actuellement dans le projet : aucun fichier `*.test.*` n'est présent dans le dépôt.
- La commande `npm test` (qui exécute `node --test`) lance 0 test : le script existe mais ne teste rien tant qu'aucun fichier de test n'est ajouté.
- Les tests rapportés ici ont été réalisés par appels HTTP temporaires, hors du projet (les fichiers du projet n'ont pas été modifiés).
- Aucun fichier de test, aucune dépendance et aucun script de test n'a été ajouté au projet.

7. Données temporaires créées puis supprimées

- Deux utilisateurs temporaires clairement identifiés (`roct.test.<timestamp>.*@test.local`) ont été créés pour les tests d'authentification et d'isolation.
- Trois produits de test temporaires (un par statut d'expiration) ont été créés puis tous supprimés via l'API.
- Nettoyage final effectué via un script temporaire (hors projet) : 2 utilisateurs, 2 notifications et 1 consommation temporaires supprimés.
- Aucune donnée existante n'a été modifiée ni supprimée : aucune `dropDatabase()`, aucune collection vidée. Les données réelles de la base sont restées intactes après les tests.

8. Conclusion

Le backend du projet **ROCT / SmartFridge** est **fonctionnel** : les 46 vérifications passent sans aucun échec ni test ignoré. L'authentification (JWT + bcrypt), la validation des entrées (Zod), la gestion des produits, des catégories (restriction admin), des notifications, le calcul des statuts d'expiration, l'appel Open Food Facts et l'isolation entre utilisateurs se comportent conformément aux comportements attendus.

La seule réserve ne concerne pas le fonctionnement de l'API mais sa **couverture de tests** : aucune suite de tests automatisés n'est encore présente dans le projet (`npm test` exécute 0 test). L'ajout d'un tel socle est recommandé comme prochaine étape pour pérenniser la validation.

ROCT · SmartFridge — Rapport de tests du backend, établi à partir des résultats réels de l'exécution des 46 vérifications (46/46 réussies).
Aucun fichier de code du projet n'a été modifié.